

Aluta — Technical Analysis & Roadmap to a Modern Social App

Prepared from a direct read of the codebase (Flutter client + FastAPI/Postgres backend). Findings are specific to what's actually in the repo, not generic advice.

1. Where Aluta stands today

Aluta is already far more than a toy. In one app you have: real-time 1:1 chat (text, images with an in-app annotation editor + captions, files, voice notes, GIFs, reactions, replies, edit, delete-for-everyone, forward, pin-a-message, read receipts, typing indicators, offline cache), presence (online / last-seen), a full local **music player** (playlist, equalizer, media-session notifications, favourites), **Listen Together** (live shared audio streaming with reconnect handling), **WebRTC voice calling**, biometric quick-unlock, and a cross-platform build (Android, Windows, Web). The backend is a clean FastAPI + Postgres service with friends, blocking, mute, and media models.

That is a *lot* of surface area for a solo build, and the UX care shows (tick colours, tail beaks, wallpaper, glassy panels, the chooser flows). The honest problem is not features — it's that the **foundation underneath the features has debt and gaps that will cap how far the app can scale and how “modern” it can feel**. The rest of this document is about that foundation and the road forward.

2. The one thing that makes Aluta different — and the strategy around it

Most apps are *either* a messenger (WhatsApp, Telegram, Signal) *or* a music app (Spotify, Apple Music). A few bolt on the other weakly. **Aluta's real, defensible identity is the fusion: chat + shared, synchronous music listening + presence**. “Listen Together” is the seed of a genuine moat — the emotional core of “let's be in the same song at the same time while we talk.”

Recommendation: stop thinking of Aluta as “a chat app that also plays music,” and start building it as “the app where you hang out *inside music with people*.” Everything differentiating below flows from leaning into that, not from out-featuring WhatsApp on messaging (you will lose that race).

3. Technical assessment — the foundation (grounded findings)

3.1 Real-time delivery is polling-first, and that won't scale

The chat currently runs a **2-second HTTP poll** (`_poll()` — `fetchMessagesBetween`) in parallel with a WebSocket. Polling every 2s per open chat is expensive on battery, data, and the

server, and it's why you hit the "messages stop after 60" bug (the poll was the real delivery path and the query was wrong).

Direction: make the WebSocket the *single source of truth* for real-time, and demote polling to a rare reconnect-reconciliation (e.g., one fetch on resume/reconnect, not a 2s loop). This is the biggest scalability lever in the app.

3.2 The main WebSocket has no authentication — this is a real security hole

/ws/{user_id} in `websocket_routes.py` accepts *any* user_id with **no token check**. Anyone who knows/guesses a user id can open that socket and receive that user's messages, typing, and call signaling. (The live WS *does* authenticate; the main one doesn't.)

Direction: require and verify the JWT on the main socket exactly like the live socket does, and reject on mismatch. Do this before any public launch — it's the most serious issue in the repo.

3.3 Listen Together relays raw audio bytes through the server, in memory

The host streams the whole song in 32 KB chunks through the FastAPI process to the listener. That works for 1:1 on Railway, but it will not scale (server CPU/RAM/bandwidth per session, no back-pressure, dies on restart) and it can't do groups.

Direction: move shared-listening to a **peer-to-peer WebRTC data/audio channel** (you already added `flutter_webrtc` for calls — reuse it) or, for rooms, a lightweight media/SFU approach. Keep the server as *signaling + sync clock only*, never the media pipe.

3.4 No push notifications — the single biggest functional gap

There is **no Firebase/FCM or APNs** anywhere. `notif_service.dart` is *local* notifications only. Consequence: incoming calls and messages only arrive while the app is foregrounded. For a social app this is disqualifying — a message app that can't wake the phone isn't a message app yet.

Direction: add FCM (Android/Web) + APNs (iOS later). This unlocks background messages, background/locked-screen **incoming calls** (with CallKit/ConnectionService), and re-engagement. Highest ROI addition you can make.

3.5 Media is served publicly, with no access control

/media is a plain static mount and image URLs load with no auth header — anyone with a URL can fetch anyone's shared image/file.

Direction: serve media behind auth or via short-lived signed URLs; scope access to conversation participants.

3.6 No end-to-end encryption

Messages and media are readable server-side. Not a bug — but privacy is a *differentiator* in 2026, and "your music hangouts are private" is on-brand.

Direction: E2E (Signal-protocol-style) for DMs is a strong medium-term play, especially paired with the “intimate, small-group” positioning.

3.7 Code structure & maintainability debt

- **Monolith files:** `chat_page.dart` ~176 KB, `home_page.dart` ~116 KB, `music_controls.dart` ~105 KB. These are very hard to change safely (you’ve felt this — staleness reverts, giant edits).
 - **Poor layering:** services live in `screens/` (`api_service.dart`, `websocket_manager.dart`, `token_helper.dart`, `user.dart` are all under `lib/screens/`). Business logic is tangled with UI.
 - **Cruft / duplication to drop:** `lib/flask_backend/app.py` (an entire *Flask* backend sitting inside the Flutter `lib/` — dead weight), duplicate `token_helper.dart` (`lib/api_service/` vs `lib/screens/`), a `theme_provider.dart` in both `lib/providers/` and `lib/screens/`.
 - **No formal state management:** everything is `setState` + a few `ValueNotifiers` + a `Provider` used only for theme. This is workable now but is why rebuilds are heavy and state bugs (stale badges, stuck buttons) keep appearing.
 - **No local database:** chat cache is JSON in `SharedPreferences` (capped at 100 messages). Fine for a cache, wrong for offline history/search at scale.
 - **No tests, no crash reporting, no CI.** Every change is validated by a 24-minute manual build. That’s the productivity ceiling you keep hitting.
 - **Toolchain drift:** Gradle/AGP/Kotlin are all one major behind (build warns), and the dependency graph is fragile (the flutter_secure_storage / SDK-37 / pub upgrade saga). You need version discipline.
-

4. What to DROP or fix (hardening — do this first)

These make everything else possible. Rough order:

1. **Authenticate the main WebSocket** (security — non-negotiable before launch).
 2. **Add push notifications** (FCM now; APNs when you do iOS) — unlocks background delivery + real incoming calls.
 3. **Demote 2s polling** to reconnect-only; make WS the real-time path.
 4. **Move Listen-Together media off the server** onto WebRTC; keep the server as signaling/sync only.
 5. **Lock down media** (auth or signed URLs).
 6. **Delete dead weight:** `lib/flask_backend/`, duplicate `token_helper`, duplicate `theme_provider`, any unused screens.
 7. **Refactor the three monoliths** into feature folders (`chat/`, `music/`, `home/`) with `widgets/`/`services` split out — do it incrementally, file by file.
 8. **Introduce Riverpod** (or Bloc) *gradually* for the messy shared state (presence, unread, now-playing, call state) — this kills a whole class of “stale UI” bugs.
 9. **Add CI + a smoke test + Sentry** so you stop discovering breakage only at build time.
-

5. What to ADD — the road to a modern social app

Grouped so you can pick a lane. The ★ items are the ones that make Aluta *different*, not just complete.

Tier A — table stakes to be taken seriously (near-term)

- Group chats (your Friend/Message model can extend to conversations/rooms).
- Push notifications (see §4).
- Media viewer polish: video messages, in-app image save (partly done), audio scrubbing.
- Global message + media search (needs the local DB).
- Stories / ephemeral status (photos, “now playing” moments).
- Account & trust: report, moderation queue, expand the existing block/mute.

Tier B — the music-social moat ★ (this is your identity)

- ★ **Listening Rooms** — many people in one synced song queue with live chat, reactions, and a shared “now playing” (think Spotify Jam × Turntable.fm × Discord stage). This is the flagship.
- ★ **“Now Playing” presence** — friends see what you’re listening to; tap to join/listen-together in one gesture.
- ★ **Collaborative playlists** — friends add/reorder in real time; the chat is the queue.
- ★ **Taste-matching / discovery** — “you and Lusajo have 78% overlap,” friend suggestions by music taste, weekly “what your circle played.”
- ★ **In-chat song reactions & song-messages** — drop a track into a thread that plays inline with a scrubber; react to the *moment* in a song.
- ★ **DJ mode / handoff** — pass control of the room’s playback around the group.

Tier C — modern-era / AI layer

- Smart replies & thread summarization (catch up on a busy room).
- Voice-note **transcription** + searchable audio.
- Music recommendations from listening + social graph (your own lightweight “Discover Weekly with friends”).
- Translation of messages inline (you already handle Swahili/English mixed threads).
- On-device moderation for safety.

Tier D — sustainability

- Premium tier (bigger rooms, HD audio, exclusive themes, longer history).
 - Creator/curator tools (public rooms, followable playlists).
 - Lightweight analytics so you know what people actually use.
-

6. A concrete phased roadmap

Phase 0 — Foundation & Trust (weeks, not features). WS auth · push notifications · WS-first delivery · media access control · delete cruft · Sentry + CI + a few tests. *Outcome: safe, reliable, launchable.*

Phase 1 — Social depth. Group chats · stories/status · search · report/moderation. *Outcome: a credible modern messenger.*

Phase 2 — The moat (music-social). Listening Rooms · now-playing presence · collaborative playlists · taste-matching. *Outcome: something no mainstream app does — a reason to switch.*

Phase 3 — Intelligence & sustainability. AI (summaries, transcription, recs) · E2E encryption · premium & creator tools. *Outcome: a defensible, modern product with a business.*

7. If you did only five things next

1. **Authenticate the main WebSocket.** (Security. Today.)
 2. **Add FCM push + background/locked-screen calls.** (Turns “demo” into “usable.”)
 3. **Make the WebSocket the real delivery path; retire the 2s poll.** (Scale + battery.)
 4. **Ship one flagship music-social feature — Listening Rooms.** (Your differentiation, built on WebRTC you already have.)
 5. **Split the three monolith files + add CI/Sentry.** (So shipping stops being painful.)
-

Closing read

You don’t have a features problem — you have a *foundation and focus* problem. Harden the base (auth, push, real-time, structure), then stop competing with messengers and go all-in on the thing they can’t easily copy: **being together inside music**. That’s Aluta’s road to the modern era.

Appendix A — Ready-to-Use Claude Prompts

Paste any of these to Claude (this session or a fresh one) to execute that task. Each is scoped to Aluta’s actual files and constraints, so Claude has the context it needs. They’re ordered by the roadmap phases. Do the **Foundation & Trust** set first.

Phase 0 — Foundation & Trust

A1 — Authenticate the main WebSocket (*security — do this first*)

In my Aluta project, the main realtime socket `/ws/{user_id}` in `my_aluta_api/websocket_routes.py` accepts any `user_id` with NO authentication — anyone can subscribe as anyone. Add JWT verification exactly like the `/live` socket already does: read the token from the query string, decode it with `SECRET_KEY/ALGORITHM`, load the user, and close the socket (code 4401) if the token is missing/invalid or its user id doesn't match `{user_id}`. Keep the

existing typing + call_* relay behaviour intact. Then update the Flutter WebSocketManager to always send ?token=<jwt> on connect, and make sure reconnect still works. Verify it compiles.

A2 — Add push notifications (FCM) (*background messages + real incoming calls*)

Add Firebase Cloud Messaging to Aluta so messages and calls arrive when the app is backgrounded or closed (right now notif_service is local-only, so nothing wakes the phone). Client: add firebase_core + firebase_messaging, register the FCM token on login and store it server-side (new device_tokens table), and handle foreground/background/terminated messages including showing an incoming-call notification. Backend (FastAPI): add an endpoint to save device tokens plus a push helper, and fire a data push on new_message and on call_offer. Add the Android Gradle/manifest config. Then walk me step-by-step through the Firebase console setup I must do myself – I have no existing Firebase project.

A3 — Make the WebSocket the real delivery path (retire the 2s poll)

In Aluta's chat_page.dart, message delivery really runs off a 2-second _poll() loop next to the WebSocket, which is a battery/scale liability and caused a past "messages stop after 60" bug. Refactor so the WebSocket is the single real-time source: new_message/edit/delete/reaction/read events update the list directly, and replace the 2s poll with ONE reconciliation fetch that runs only on (re)connect and on app-resume. Keep the offline cache fallback and make sure read receipts + delivered ticks still work. Explain any case where a message could be missed and how the reconnect fetch covers it.

A4 — Move Listen-Together audio off the server (WebRTC)

Aluta's Listen Together (my_aluta_api/live_session.py + live_session_service.dart) streams the whole song through the FastAPI server in 32KB chunks – it won't scale. Re-architect it to send audio peer-to-peer over a WebRTC data

channel
(reuse the flutter_webrtc setup in call_service.dart), keeping the server only as signaling + a sync clock. Preserve the UX: host picks a track, listener buffers and plays, play/pause/seek stay in sync, and the reconnect grace-window still works. Keep it 1:1 for now but design the signaling so it can extend to rooms.

A5 — Lock down media access

In Aluta, uploaded media is served by a public StaticFiles mount at /media with no auth, and images load with no token – any URL is world-readable. Change media delivery so only conversation participants can fetch a file: either an authenticated endpoint that checks the requester is the sender/receiver, or short-lived signed URLs. Update the Flutter image/file loaders accordingly, and migrate existing messages gracefully so nothing breaks.

A6 — Delete dead code & duplication

Clean up Aluta's repo cruft with NO behaviour change: remove lib/flask_backend/ (a stray Flask backend sitting inside the Flutter lib), de-duplicate token_helper.dart (lib/api_service/ vs lib/screens/) and theme_provider.dart (lib/providers/ vs lib/screens/) to one canonical copy each with imports fixed, and report any other unused files. Run flutter analyze afterwards and confirm no new errors.

A7 — Break up the monolith files

chat_page.dart (~176KB), home_page.dart (~116KB) and music_controls.dart (~105KB) are too big to maintain safely. Refactor ONE of them (start with chat_page.dart) into a feature folder: pull the message bubble, the composer/footer, the image editor, and the pin/forward/call helpers into their own widget/service files, leaving chat_page as a thin coordinator. Pure refactor, no behaviour change – bracket-check and confirm it still analyzes clean.

A8 — Introduce Riverpod for shared state

Aluta relies on `setState` + scattered `ValueNotifiers`, which keeps causing stale-UI bugs (unread badges, presence, now-playing, call state). Add Riverpod and migrate ONE cross-cutting piece first – the now-playing / live-session state – into a provider that replaces the ad-hoc notifiers, keeping everything working. Show the before/after and explain the pattern so I can repeat it for presence and unread counts.

A9 — Add CI, a smoke test, and crash reporting

Set up a GitHub Actions workflow for Aluta that runs ``flutter analyze`` and ``flutter test`` on every push and builds a debug APK artifact. Add a minimal widget smoke test that boots the app to the login screen. Add `sentry_flutter` with `init` in `main.dart` (DSN placeholder) capturing Flutter + async errors. Keep it lightweight.

Phase 1 — Social depth

B1 — Group chats

Extend Aluta from 1:1 to group conversations. Backend: add `conversations` + `members` models (a conversation has many users; messages belong to a conversation), migrate the existing 1:1 messages into 2-member conversations, and update the `send/fetch/read` endpoints and WS fan-out to target a conversation's members. Client: group creation, a group chat screen (reuse `chat_page`), member avatars, and sender labels on messages. Keep DMs working unchanged.

B2 — Stories / status

Add ephemeral Stories to Aluta: a user posts a photo/short clip or a "now playing" moment that expires in 24h. Backend: a `stories` table with `expiry` and a `viewed-by` list, auto-expiring server-side. Client: a stories tray atop the Messages list, a full-screen viewer with tap-through and progress bars, and a "who viewed" list.

B3 — Global message & media search

Add search to Aluta. Add a local database (drift or isar) so the client can index messages for fast offline search, plus a backend endpoint for older history. UI: a search field in the Messages header that finds messages by text and filters media (photos/files/voice/GIF), highlights matches, and taps-to-jump to the message in the thread.

Phase 2 — The music-social moat ★

C1 — Listening Rooms (flagship)

Build Aluta's flagship feature: Listening Rooms – a room where several people share ONE synchronized music queue with live chat and reactions. Reuse the Listen-Together sync engine but for N participants: a host/DJ controls playback, everyone hears the same position, anyone can add to the queue, and a live chat + emoji-reaction rail sits beside the player. Backend: room + membership + queue models and WS fan-out. Give me a phased plan FIRST (data model, sync strategy, where WebRTC/SFU replaces server streaming for scale), then build phase 1.

C2 — “Now Playing” presence

Add now-playing presence to Aluta: while a user plays music, broadcast their current track to friends. Show it under their name in the friends list and chat header ("♪ listening to X") and add a one-tap "Listen with them" that starts a Listen-Together session. Respect a privacy toggle to hide it.

C3 — Collaborative playlists

Add collaborative playlists to Aluta where a group builds a shared queue in real time. Backend: playlist + tracks + collaborators models with WS updates on add/remove/reorder. Client: a shared playlist screen that live-updates as others edit, drag-to-reorder, and a "play this together" action that opens it as a Listening Room.

C4 — Music taste-matching & discovery

Add taste-matching to Aluta. Track what each user listens to, compute listening overlap between friends, and surface it: a "you and X share 78% taste" chip, friend suggestions by music overlap, and a weekly "what your circle played" digest. Keep it lightweight – server-side aggregation, no heavy ML to start.

Phase 3 — Intelligence & sustainability

D1 — AI assist (smart replies, summaries, transcription)

Add an AI layer to Aluta via an LLM API: (1) smart-reply suggestions above the composer, (2) "catch me up" thread/room summarization, and (3) voice-note transcription with searchable text. Put all model calls behind a backend endpoint (keys stay server-side), add simple caching, and make each feature toggleable. Start with voice-note transcription.

D2 — End-to-end encryption for DMs

Add end-to-end encryption to Aluta's direct messages (Signal-protocol style): per-device key generation, a key/prekey exchange through the backend, and encrypt/decrypt on the client so the server only stores ciphertext. Handle multi-device and key rotation. Give me a design doc + threat model FIRST, then implement the key exchange and message encryption incrementally.

D3 — Premium tier & analytics

Add a lightweight premium tier and product analytics to Aluta. Premium unlocks bigger Listening Rooms, HD audio, longer history, and exclusive themes, gated by a server-side entitlement flag (wire store billing later). Add privacy-respecting analytics (screen views + key events) so I can see what's actually used.

Tip: run these roughly in order. Anything in Phase 0 is safe to hand over immediately; Phase 2's C1 (Listening Rooms) is the one worth doing well, since it's what makes Aluta different.